

	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERIA ESCUELA DE COMPUTACIÓN
CICLO 2	GUIA DE LABORATORIO #6 Programación Orientada a Objetos (POO404) Creación de Interfaces gráficas con SWING

I. OBJETIVOS

Que el estudiante:

- Cree interfaces gráficas con SWING utilizando las herramientas de Netbeans.
- Maneje diferentes eventos de los componentes gráficos de Swing, según sea la necesidad.

II. INTRODUCCION

Diseño de GUI con Java

Java posee un juego de clases capaces de construir potentes *interfaces gráficas de usuarios* o **GUI** (Graphical user Interface). A este grupo de clases se les denomina **JFC** (Java Foundation Classes) y presentan las siguientes características:

- Diseño de interfaces puramente Java y por lo tanto independientemente de la plataforma donde se ejecute.
- Extensible y personalizable: se trata de un framework intuitivo, ampliable por el programador, extendiendo e implementando las clases base. Es totalmente personalizable a nivel de look and feel, siendo posible aplicar diferentes tipos de apariencias de las ventanas, en principio, independientemente del sistema operativo sobre el que se ejecute.

La interfaz gráfica de java posee los siguientes elementos comunes:

- Contenedores: se pueden distinguir dos tipos: los contenedores base, que representan al marco de la ventana y son necesarios al menos uno por aplicación, y los contenedores que agrupan a los componentes, considerando este último tipo de contenedor como un componente que agrupa otros componentes bajo un mismo panel.
- Componente: controles que se insertarán en los contenedores (botones, cajas de texto...).
- Layouts: representan la organización de los controles sobre un contenedor.
- Gestión de eventos: Mecanismo por el cual es posible capturar y procesar los eventos del usuario producidos sobre el contenedor y sus controles.

¿Qué es Swing?

Swing es un conjunto de paquetes con los que cuenta Java para crear y mostrar una interfaz gráfica de usuario.

Es una implementación del entorno gráfico 100% basada en Java y su apariencia de visualización no es dependiente del entorno (Sistema operativo).

Los paquetes de Swing contienen diversidad de clases para generar ventanas, mensajes, botones, cajas de texto e incluso imágenes, audio o video.

Swing forma parte del paquete estándar, no hace falta importar ningún fichero adicional en el proyecto que sea externo de la API que ya tiene el lenguaje. No se trata de un nuevo framework de desarrollo en sistemas de ventanas, sino que más bien se basa en AWT, cuya aparición se dio en la versión 1.2 de Java.

La librería está implementada en 18 paquetes, de los cuales, los más utilizados son los siguientes:

- `javax.swing`: es el paquete de más alto nivel, que contiene los componentes, adaptadores, los modelos por defecto de los componentes, y las interfaces para todos los modelos.
- `javax.swing.event`: contiene los tipos de eventos y listeners (oyentes) específicos de Swing. Además, los componentes Swing pueden generar sus propios eventos.

Una de las clases más importantes de Swing es **JFrame**. Esta clase genera una ventana estándar de Windows, la cual tiene un título, bordes y controles de ventana (minimizar, maximizar, cerrar).

La clase Swing consta de un contenedor denominado **ContentPane**, en el que podemos poner los elementos (controles o componentes) de la interfaz gráfica.



JFrame: Controles

La clase JFrame consta de un conjunto de herramientas de tipo Control que permiten generar acciones o eventos dentro de una aplicación con GUI.



- **Etiqueta (jLabel):** un área de visualización de una cadena de texto o una imagen, o ambas.
- **Botón (jButton):** un botón "oprimir"
- **Botón de 2 posiciones (jToggleButton):** un botón con dos estados.
- **Casilla de activación (jCheckBox):** un elemento puede ser seleccionado o deseleccionado. Por convención, cualquier jCheckBox en un grupo puede ser seleccionado.
- **Botón de opción (jRadioButton):** un elemento puede ser seleccionado o deseleccionado. Usado con un objeto buttonGroup para crear un grupo de botones en el cual un botón puede ser seleccionado a la vez.
- **Grupo de botones (buttonGroup):** esta clase se utiliza para crear un conjunto de botones en que sólo se podrá seleccionar uno.
- **Lista desplegable (jComboBox):** un componente que combina un botón o campo editable y una lista desplegable.
- **Lista (jList):** un componente que permite seleccionar uno o más objetos de una lista.
- **Campo de texto (jTextField):** un componente ligero que permite editar una línea individual de texto.
- **Área de texto (jTextArea):** un área de varias líneas que muestra textos sin adornos.
- **Barra de desplazamiento (jScrollBar):** un componente que permite ajustar el contenido del área visible de otro componente.
- **Deslizador (jSlider):** un componente que permite seleccionar gráficamente un valor al deslizar un tirador dentro de un intervalo limitado.
- **Barra de progreso (jProgressBar):** informa habitualmente del progreso de algún trabajo, mostrando un porcentaje de realización y quizás una visualización textual de este porcentaje.
- **Cuadro formateado (jFormattedTextField):** un componente que permite editar un valor formateado (línea de texto), así como recuperar un objeto particular una vez que se ha editado el texto.
- **Cuadro de contraseña (jPasswordField):** un componente que permite editar una línea individual de texto donde se indica que algo se ha tecleado, pero no muestra los caracteres originales.
- **Spinner (jSpinner):** cuadro de entrada de una línea individual que permite seleccionar un número o un valor de una secuencia ordenada.
- **Separador (jSeparator):** un componente de utilidad diversa para realizar líneas divisorias
- **Panel de texto (jTextPane):** un componente de texto que se puede marcar con atributos que se representarán gráficamente.
- **Panel editor (jEditorPane):** un componente de texto para editar diferentes contenidos
- **Árbol (jTree):** un control que muestra un conjunto de datos jerárquicos como un boceto.
- **Tabla (jTable):** componente utilizado para mostrar y editar las habituales tablas bidimensionales con celdas.

Contenedores de Controles (JPanel)

Las ventanas de control JFrame contienen un contenedor y dentro de este, se le debe agregar un panel de contenido (Clase JPanel), en el cual se deben incluir todos los controles que se vayan a utilizar.

Por lo tanto, antes de incluir cualquier control, estos necesitaran estar incluidos dentro del panel.

Ademas de este aspecto, tambien se puede incluir un gestor de organizacion llamados Layout para la distribucion de los controles.

Prefijos de Controles

En cualquier lenguaje de programacion, incluido Java, que implementa GUI, necesita identificar a cada Control incluido en las ventanas y diferenciar a la clase utilizada en cada Controles.

Los prefijos son convenciones de nomenclatura utilizadas para identificar la *clase* de un control o componente en una interfaz gráfica. Aunque no son parte de la sintaxis del lenguaje, ayudan a la legibilidad y mantenimiento del código orientado a GUI.

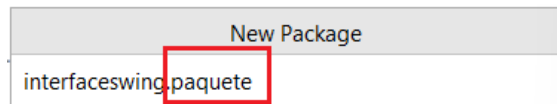
Prefijos comunes para controles Swing en Java:

btn: Para botones (JButton)	txt: Para campos de texto (JTextField, JTextArea)
chk: Para cajas de selección (JCheckBox)	pwd: Para campos de contraseñas (JPasswordField)
cmb: Para listas desplegables (JComboBox)	pgb: Para barras de progreso (JProgressBar)
lbl: Para etiquetas (JLabel)	mnu: Para menús (JMenu)
lst: Para listas (JList)	frm: Para ventanas (JFrame)
pnl: Para paneles (JPanel)	dlg: Para diálogos (JDialog)

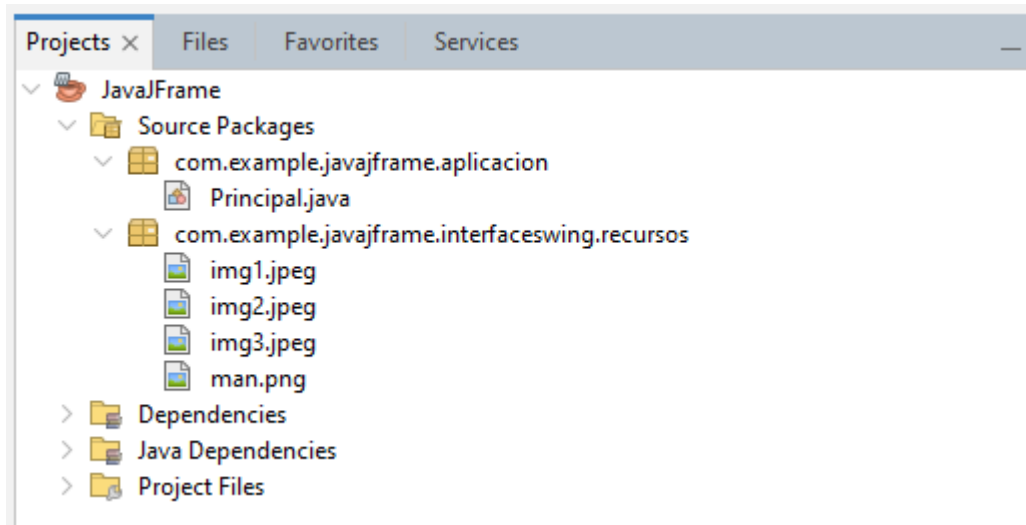
III. PROCEDIMIENTO

PARTE 1: : Creación de GUI con el diseñador de interfaces de usuario del IDE

1. Antes de iniciar, descargue los recursos adjuntos proporcionados para el procedimiento de esta Guia de Practica.
2. Ahora, desde Netbeans, cree un nuevo proyecto java con el nombre **POOGuia6**.
3. En la carpeta **src**, cree a 2 paquetes llamados **interfaceswing** y **aplicacion**,
4. En el paquete **aplicacion** cree a una clase llamada **Principal**. En la definición de esta clase, redacte al método estático de inicio **main**.
5. Dentro del paquete **interfaceswing**, cree a un paquete llamado **recursos**. Solo escriba la palabra recursos al final del texto del cursor ya ofrecido en la opción:

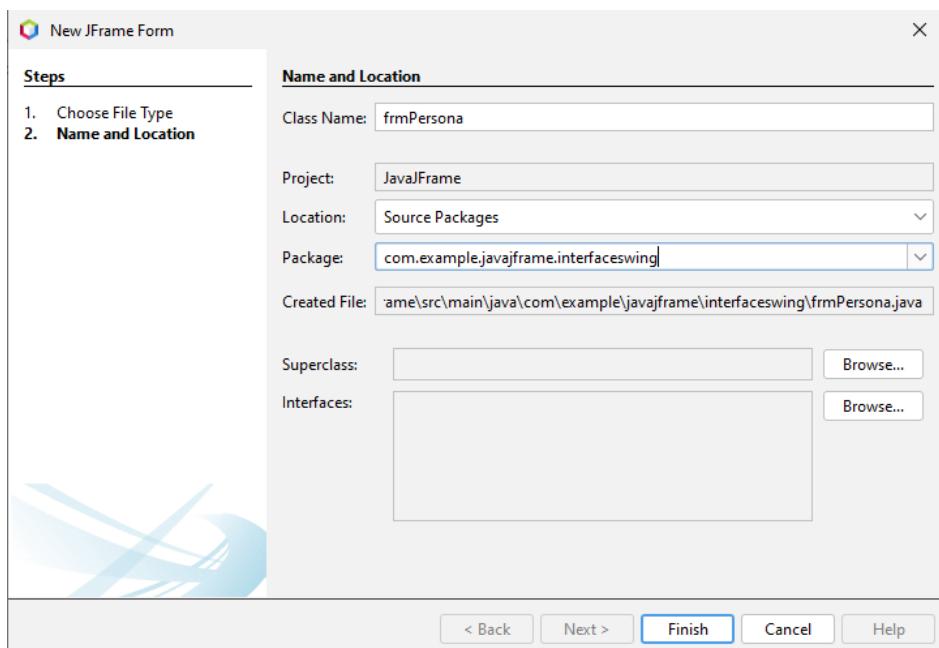


6. Con el explorador de Windows, abra la carpeta con los archivos complementarios proporcionados para esta práctica. Seleccione a todos los archivos, haga clic secundario y seleccione la acción *Copiar*.
7. Retorne al proyecto java y haga clic secundario sobre el paquete **recursos**. Seleccione la acción *Pegar*. Haga clic en OK:

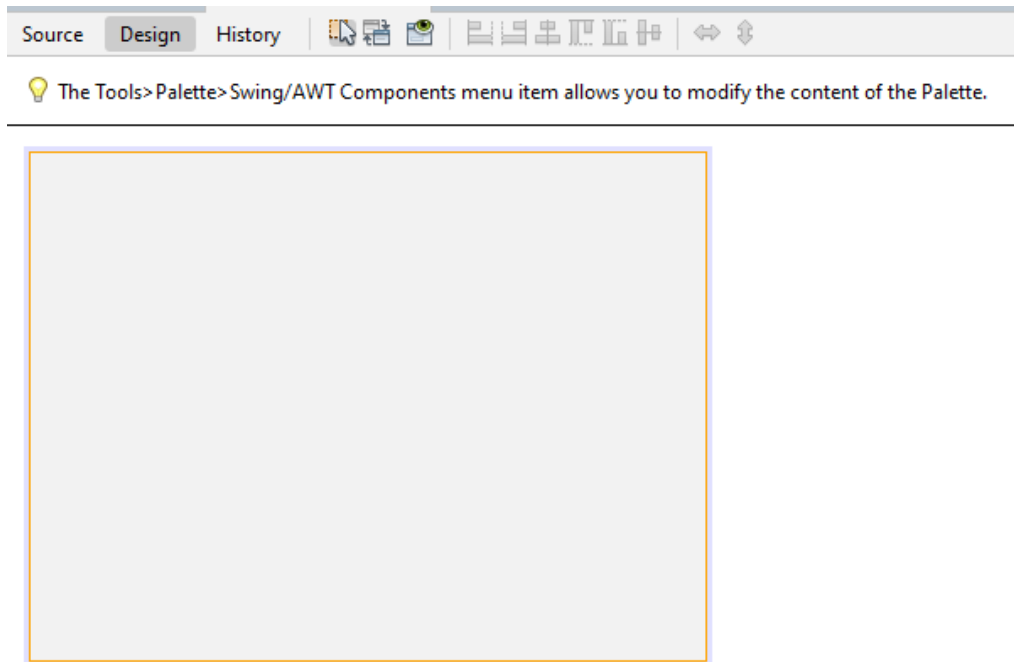


Haga clic en botón Aceptar y verifique que todos los archivos proporcionados se copiaron en el paquete indicado.

8. Haga clic secundario en el paquete **interfaceswing**. Del menu contextual, seleccione a las opciones *new* -> *JFrameForm*.



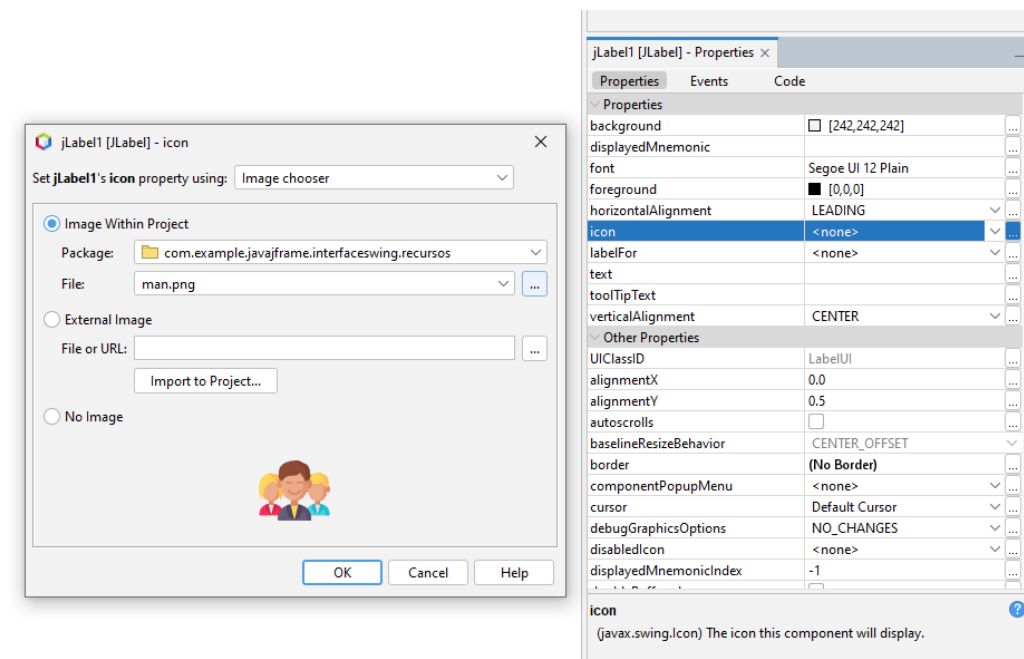
9. Se abre una ventana para editar al formulario y a los controles que serán ubicados en su contenedor. El formulario se define en 3 archivos distintos (solo utilizaremos Source y Design).



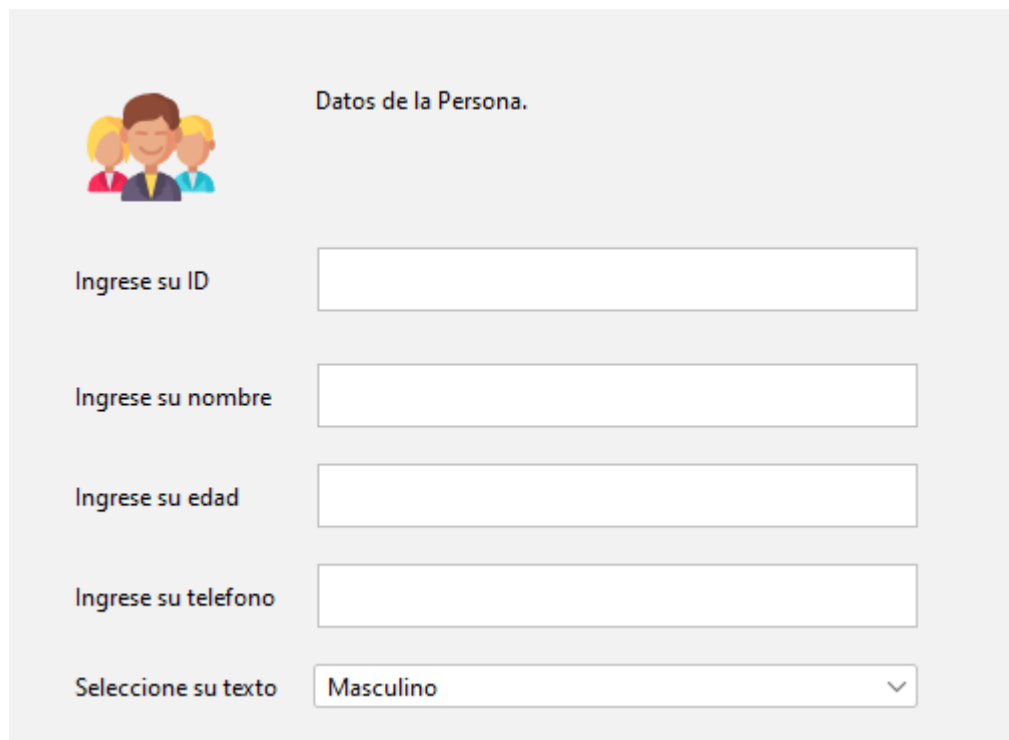
10. Agregue un Label al formulario De la *lista de propiedades* de este objeto de la clase Label incluido, ubique a la propiedad **text** y borre su valor actual.

Luego, busque la propiedad **icon** y haga clic en el icono de carpeta de esta propiedad.

En la ventana de selección de archivos de icono, expanda a la carpeta **src** y localice la ruta del archivo de imagen (*interfaceswing/recursos/man.png*) y haga clic en OK.



11. Agregar un nuevo Label y ubicarlo a la derecha de la imagen ya existente en el form. Luego modificar su propiedad **text** con el texto Datos de la Persona.
12. Con el uso de la herramienta Pallette, utilice los controles de Label, Text Field, y Combo Box, para crear la siguiente interfaz.



The screenshot shows a Java Swing window titled "Datos de la Persona." with a light gray background. At the top left is an icon of three people. Below the title, there are five input controls: four text fields and one dropdown menu. The labels for these controls are "Ingrese su ID", "Ingrese su nombre", "Ingrese su edad", "Ingrese su telefono", and "Seleccione su sexo". The dropdown menu currently shows "Masculino".

13. Cuide de colocar a cada uno de los controles los textos de la imagen. El ultimo control es un *ComboBox*.
14. Para que la interfaz se vea igual que la imagen del paso anterior, proceda a seleccionar a cada control listado a continuación y desde la ventana **Component Tree**, modifique las propiedades especificadas ahí: (Variable Name esta en la pestaña de Code)

Control	Propiedad	Valor	Control	Propiedad	Valor
JPanel	Accesible Name	pn1Persona	JTextField1	Variable Name	txtId
JLabel1	text	Ingrese su id:	JTextField2	Variable Name	txtNombre
JLabel2	text	Ingrese su nombre:	JTextField3	Variable Name	txtEdad
JLabel3	text	Ingrese su edad:	JTextField4	Variable Name	txtTelefono
JLabel4	text	Ingrese su teléfono:	JComboBox1	Variable Name	cmbSexo
JLabel5	text	Selee su sexo:		model	Masculino Femenino

15. Haga las acciones necesarias para quitar los textos de los textFields.

16. De la paleta de controles, seleccione un **JPanel** y ubíquelo debajo de los controles agregados hasta el momento
17. Seleccione dos controles **Button** y ubíquelos al centro del nuevo JPanel .

The screenshot shows a Java Swing window titled "Datos de la Persona." with a light gray background. At the top left is an icon of three people. Below the title, there are five input fields arranged vertically, each with a label to its left: "Ingrese su ID", "Ingrese su nombre", "Ingrese su edad", "Ingrese su telefono", and "Seleccione su sexo". The "Seleccione su sexo" field is a dropdown menu currently showing "Masculino". At the bottom of the form are two buttons: "Limpiar" and "Obtener Datos".

18. Configure las siguientes propiedades de los últimos controles (el JPanel y los 2 JButton):

Control	Propiedad	Valor
JPanel	Accesible Name	pnlBotones
JButton1	Variable Name	btnObtenerDatos
	text	Obtener Datos
JButton2	Variable Name	btnLimpiar
	text	Limpiar

19. Hasta este punto solamente se ha creado el diseño preliminar, pero no se puede correr la aplicación hasta que esta clase sea invocada desde el método de inicio **main**.
20. Ingrese a la ficha de edición del archivo (Souce . java). Observe que la clase **frmPersona** contiene declarados como campos a los Controles que usted modificó la propiedad *Accesible name* en la ventana de diseño.

21. Al inicio del código, importe a la clase `java.awt.Dimension`.

22. La clase `frmPersona` herede de la clase **JFrame**:

```
public class frmPersona extends JFrame {
```

23. Proceda a modificar el constructor con parámetros, el cual tendrá los datos de inicialización del `JFrame`:

```
public frmPersona(String title) {  
    super(title);  
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    this.setMinimumSize(new Dimension(600, 500));  
    this.setLocationRelativeTo(getParent());  
    initComponents();  
}
```

24. Para ejecutar el form, abra la definición de la clase **Principal** y ubique a la definición del método **main**.

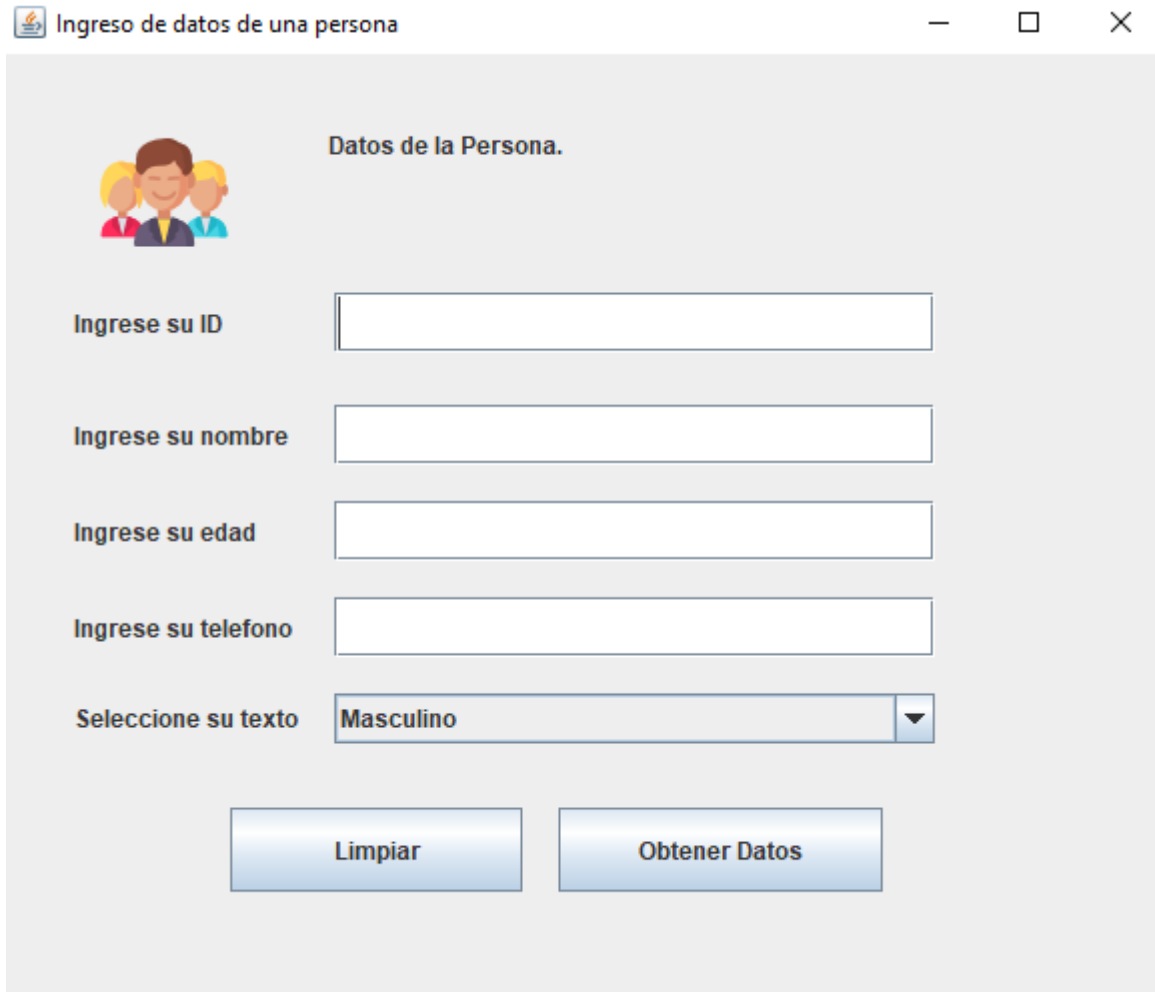
25. Importe el paquete de `interfacesswing` en la clase `Principal` y digite el siguiente segmento de código para declarar, instanciar un objeto de la clase `frmPersona` y verlo en pantalla:

```
frmPersona formpersona = new frmPersona(  
    "Ingreso de datos de una persona");  
  
formpersona.setVisible(true);
```

26. En el archivo `pom.xml` hay que cambiar la propiedad `<exec.mainClass>` por el nombre de nuestro paquete `Principal` (`com.example.javajframe.aplicacion.Principal`).



27.Proceda a ejecutar a su aplicación. El resultado sera el siguiente:

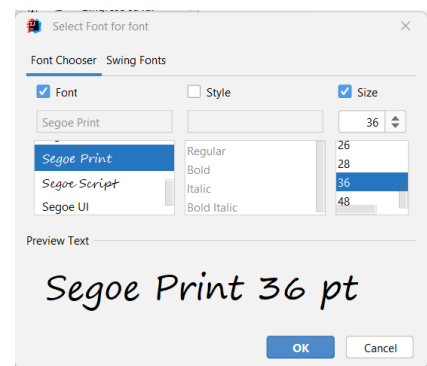


28.Toque el botón de Cerrar(x) de la ventana y así retornar al modo de diseño.

29.Ahora cambiara un poco al diseño del form. Seleccione al JLabel que tiene el título “Datos de la Persona”.

Luego, de la lista de sus propiedades, ubique a la propiedad **Font**. Cambie el tipo de letra y tamaño a los mostrados en la imagen a continuación.

Haga clic en boton Ok.



30.Mantenga seleccionada la tecla Ctrl y haga clic sobre cada uno de los demás controles (JLabel, JTextField y JButton), excepto JLabel que usted modifico en el paso anterior.

Deje de presionar la tecla Ctrl. Luego, de la lista de propiedades, seleccione la propiedad **Font**.

En la ventana de configuración del tipo de letra, marque el mismo tipo de letra usado (*Segoe Print*), seleccione en **Style: Bold** y **Size: 16**.

31. Vuelva a ejecutar su aplicación y observe el cambio en la interfaz:



 **Datos de la Persona.**

Ingrese su ID

Ingrese su nombre

Ingrese su edad

Ingrese su telefono

Seleccione su texto Masculino 

32. Cierre el form y así retorne al modo de diseño.

33. Ahora se procederá a darle funcionalidad al par de botones incluidos en el form.

34. Vaya a la clase **frmPersona**. Luego del cierre de la definición de su método constructor, redacte al siguiente par de métodos:

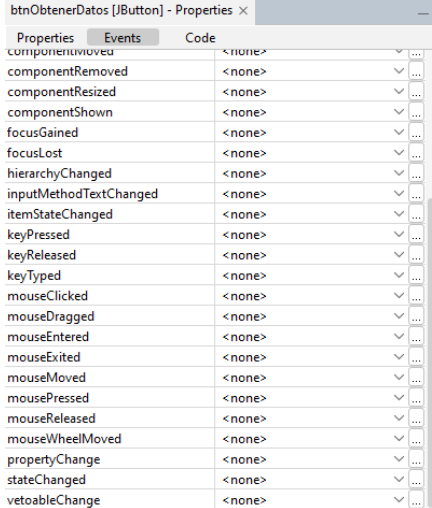
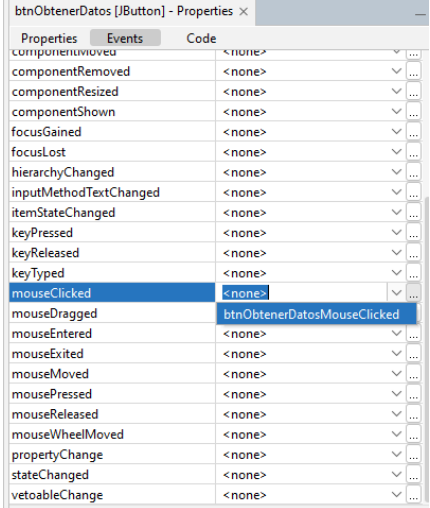
```
private void btnObtenerDatos() {  
    String id;  
    String nombres;  
    String edad;  
    String telefono;
```

```
String sexo;
//adquiere de controles del form a datos dados por usuario
id = txtId.getText();
nombres = txtNombre.getText();
edad = txtEdad.getText();
telefono = txtTelefono.getText();
sexo = cmbSexo.getSelectedItem().toString();

JOptionPane.showMessageDialog(null,
    "Datos Obtenidos: \n ID: " +id+
    "\n nombres: "+ nombres+"\n Edad: "+edad+
    "\n Telefono: "+telefono+ "\n Sexo: "+ sexo);
}

private void btnLimpiar() {
    //borra contenido actual de Los controles del form
    txtId.setText("");
    txtNombre.setText("");
    txtEdad.setText("");
    txtTelefono.setText("");
    //pone el combobox en su valor de índice 0
    cmbSexo.setSelectedIndex(0);
    txtId.requestFocus(); //recibe el foco
}
```

35. Haga clic secundario sobre el botón **btnObtenerDatos**. Del menú de propiedades cambie a la pestaña de Events, seleccione el evento MouseClicked. Tal y como se muestra en la siguiente imagen, aparecerá un listado con los diferentes *Listener* a utilizar, para este caso se deberá seleccionar el que aparece por defecto.

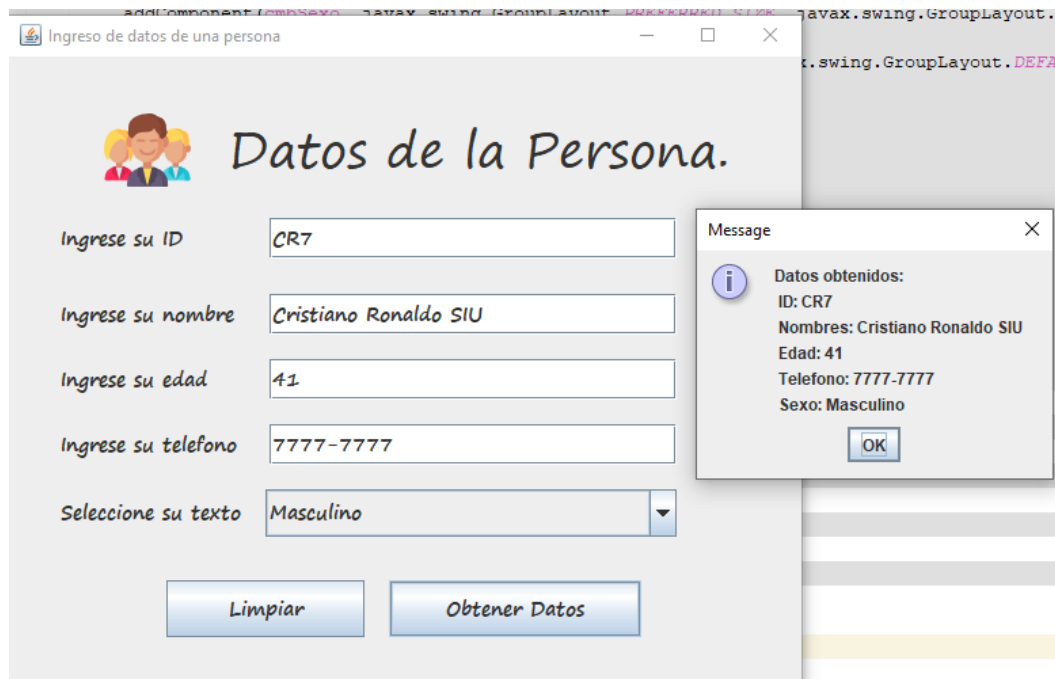
Selección de Create Listener	Tipos de Listener
	

36. Se creará automáticamente el método de `btnObtenerDatosMouseClicked`, en este evento mande a llamar las funciones creadas anteriormente.

```
private void btnObtenerDatosMouseClicked(java.awt.event.MouseEvent evt) {
    btnObtenerDatos();
}

private void btnLimpiarMouseClicked(java.awt.event.MouseEvent evt) {
    btnLimpiar();
}
```

37. Vuelva a ejecutar a su aplicación. Ingrese datos en los cuadros de texto y el combo, para presionar el botón con el texto *Obtener Datos*. El resultado sera similar al mostrado a continuación:



38. Haga clic en la ventana de dialogo JOptionPane abierta y luego en su botón Cerrar(X), para retornar al modo de diseño.

PARTE 3: Creación de un Modelo de Tabla

Para esta parte se utilizarán los model para poder dibujar una tabla. Los modelos de tabla son objetos que implementan la interface **TableModel**; a través de ellos es posible personalizar mucho más y mejor el comportamiento de los componentes llamados **JTable**, permitiendo utilizar al máximo sus potencialidades. Todas las tablas cuentan con un modelo de tabla, existe uno por omisión.

El siguiente gráfico intenta mostrar cómo cada componente JTable obtiene siempre sus datos desde un modelo de tabla.



Relación Modelo -> Vista

La clase **AbstractTableModel** es la que implementa directamente a la interface **TableModel**, aunque es esta clase la que se recomienda extender para utilizarla como modelo de tabla, existe un modelo de tabla predeterminado que facilita mucho el trabajo con tablas.

Este modelo predeterminado es la clase **DefaultTableModel**.

39. Al final del form, inserte un **JScrollPane** y ubíquelo debajo de los botones.
40. Y dentro de este nuevo control **JScrollPane** insertado, ubique a un control **JTable**.

41. De este último control JTable, modifique a su propiedad **variable name** por **tblDatos**.

42. Retorne a la definición de la clase **frmPersona**.

43. Del listado de campos ya existentes en esta clase, declare al siguiente campo, el cual será de la clase **DefaultTableModel**:

```
private DefaultTableModel modelo;
```

44. Ahora ubique la declaración del método constructor de esta clase.

Ahí agrugue el siguiente código justamente después de la función **initComponents**:

```
//Arreglo de objeto, para inicializar con vacio la tabla
Object [][] data=null;
//Arreglo de String para crear los nombres de las columnas
String[] columns= {
    "Id", "Nombres", "Edad", "Telefono", "Sexo"
};
//Instancia del modelo
modelo= new DefaultTableModel(data, columns);

//Seteo del modelo, el cual tendra la estructura que permitira
// a la tabla representar los datos
tblDatos.setModel(modelo);
```

45. Para poder agregar los datos capturados desde el form hacia la tabla **tblDatos**, será necesario incluir el siguiente código al final del código actual del método **btnObtenerDatos**:

```
//toma variables con datos recibidos del usuario
//y crea un vector genérico (tipo Object)
Object[] newRow = {id, nombres, edad, telefono, sexo};
//agrega contenido del vector como una fila de tabla modelo
modelo.addRow(newRow);
```

46. Ejecute nuevamente a su aplicación. Ingrese los datos y verá que luego del **JOptionPane** de verificación de la captura de datos, estos se agregan a la JTable **tblDatos**, similar a la pantalla siguiente:



Datos de la Persona.

Ingrese su ID

Ingrese su nombre

Ingrese su edad

Ingrese su telefono

Seleccione su texto

Limpiar

Obtener Datos

Id	Nombres	Edad	Telefono	Sexo
1	Kevin	24	7605-5255	Masculino

47.Retorne a la definici3n de la clase **frmPersona**. Cree al siguiente m3todo llamado **tblObtenerFila**, el cual se ejecutar3 cuando en el objeto **tblDatos** se desencadene el evento **MouseClicked**:


```

private void tblObtenerFila(MouseEvent e) {
    //obtiene num. fila y columna de celda de tabla en
    //donde usuario hizo clic
    int fila = tblDatos.rowAtPoint(e.getPoint());
    int columna = tblDatos.columnAtPoint(e.getPoint());

    //traslada datos de fila hacia controles del form,
    //solo si tabla ya tiene filas
    if ((fila > -1) && (columna > -1)){
        txtId. setText(modelo.getValueAt(fila, 0).toString());
        txtNombre. setText(modelo.getValueAt(fila, 1).toString());
        txtEdad. setText(modelo.getValueAt(fila, 2).toString());
        txtTelefono. setText(modelo.getValueAt(fila, 3).toString());
        cmbSexo.setSelectedItem(modelo.getValueAt(fila, 4).toString());
    }
}

```

48. Retorne a la vista del form **frmPersona**.

Ahora se le dará una funcionalidad adicional a su tabla **tblDatos**, creándole un método para su evento **MouseClicked**. Este evento se activa/dispara cuando usuario hace clic sobre cualquier celda de sus filas de información presentada.

```

private void tblDatosMouseClicked(java.awt.event.MouseEvent evt) {
    tblObtenerFila(evt); // TODO add your handling code here:
}

```

49. Ejecute nuevamente su aplicación e ingrese los datos de 3 personas.

Luego, haga clic en cualquier celda de la fila 2 de su tabla **tblDatos**. Los datos de esta fila serán trasladados a los controles del form, dando así la oportunidad de implementar un mantenimiento de datos específicos de esta persona, sin tener que volver a ingresar todos sus datos nuevamente.

50. Haga clic en celdas de otras personas en la tabla. En cada prueba, los datos completos de una fila son trasladados a los Controles de entrada del form.

51. Cierre el form, para así retornar al modo de diseño de la clase **frmPersona**.

52. Para finalizar este ejemplo, solo falta modificar la funcionalidad al insertar un nuevo registro, de manera tal que los controles **JTextField** y **JCombo** queden vacíos, listos para redactar a un nuevo registro.

53. Dentro del método **btnObtenerDatos**. En su definición actual, incluya al final a la llamada del método **btnLimpiar()**, así:

```

private void btnObtenerDatos () {
String id;
String nombres;
String edad;
String telefono;
String sexo;

id=txtID.getText();
nombres=txtNombre.getText();
edad=txtEdad.getText();
telefono=txtTelefono.getText();
sexo=cmbSexo.getSelectedItem().toString();

JOptionPane.showMessageDialog(null, "Datos obtenidos: \n ID: "+id+
    "\n Nombres: "+nombres+
    "\n Edad: "+edad+
    "\n Telefono: "+telefono+
    "\n Sexo: "+sexo);
Object[] newRow = {id, nombres, edad, telefono, sexo};
//agrega contenido del vector como una fila de tabla modelo
modelo.addRow(newRow);
btnLimpiar();
}

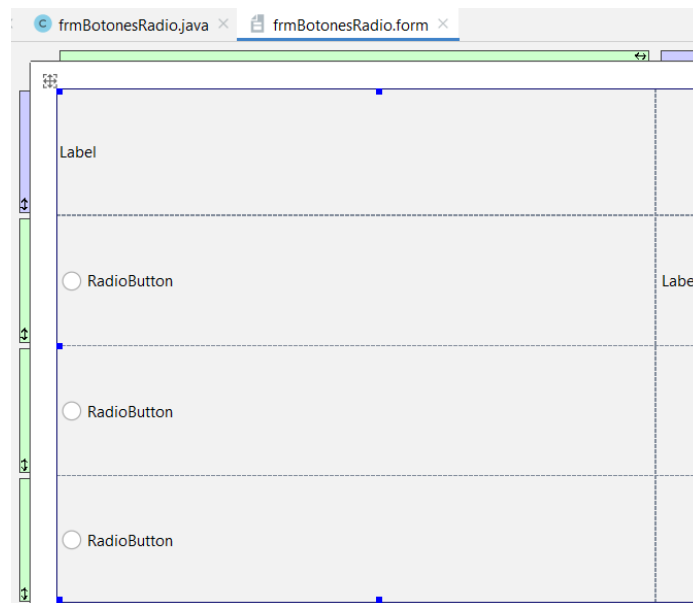
```

54. Vuelva a ejecutar su aplicación e ingrese datos de prueba. Vera que al completar los datos de una persona y presionar el boton para obtener datos, estos se borran de los controles textbox y combobox, dejando la interfaz lista para crear otro registro.

55. Cierre el formulario y retornar al modo diseño. Guarde los cambios realizados hasta ahora.

PARTE 4: Demostración de uso de la clase JRadioButton (IntelliJ IDEA)

56. En el último ejemplo de este procedimiento, utilizara a objetos JRadioButton para que usuario elija una imagen a mostrar.
57. Localice la ventana del diseñador de carpetas de su proyecto. Dentro del paquete **interfaceswing**, cree a un nuevo objeto JFrame, bajo el nombre **frmBotonesRadio**.
58. Divida el área de diseño del form en un conjunto de 4 filas x 2 columnas.
- Luego, ubique a 2 controles JLabel y a 3 JRadioButton, en las posiciones mostradas a continuación:



59. Desde la ventana **Component Tree**, modifique a las propiedades de los controles listados en la siguiente tabla:

Control	Propiedad	Valor
JPanel	Field name	pnlImagenes
JLabel1	text	Seleccione una imagen
	font	Font:Segoe Print, Style:Bold, Size:22 pt
JLabel2	text	(vacio)
	field name	lblImagen
	icon	interfaceswing/recursos/question-icon.jpeg
JRadioButton1	text	Opcion 1
	field name	rbtOpcion1
JRadioButton2	text	Opcion 2
	field name	rbtOpcion2
JRadioButton3	text	Opcion 3
	field name	rbtOpcion3

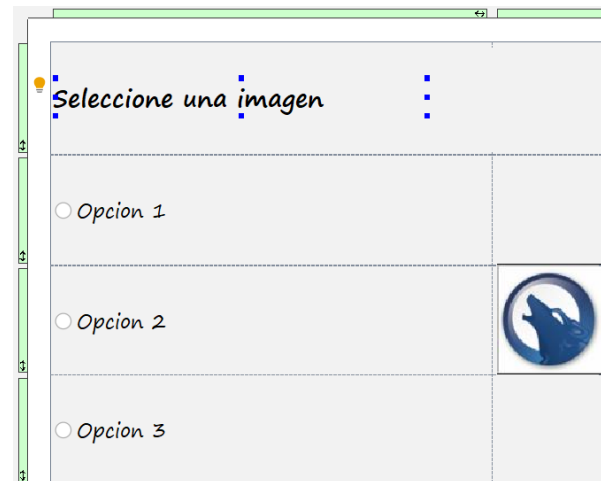
60. Seleccione a los 3 JRadioButton. Para hacerlo, mantenga presionada la tecla **Ctrl**, mientras hace clic a este trio de Controles específicos. Luego, modifique al siguiente par de propiedades comunes a estos controles:

Propiedad	Valor
buttonGroup	buttonGroup1
font	Font: Segoe Print, Size: 18 pt

61. Haga clic en el JLabel lblImagen. Luego, coloque el cursor del ratón en una de las esquinas inferiores de este control y arrástrelo verticalmente hacia abajo hasta alcanzar la última fila. Las 3 celdas verticales se fundirán en una sola celda.

62. De manera similar al paso anterior, seleccione a la JLabel de la primera fila y arrastre la esquina derecha de manera horizontal hasta fundir la celda con las 2 columnas.

63. El diseño final se verá idéntico al de la siguiente imagen:



64. Ir al código fuente de la clase **frmBotonesRadio**. Modifique su encabezado para que esta clase se derive de la clase JFrame.

65. Importe a la clase `java.awt.Dimension`.

66. Defina al siguiente método constructor para esta clase:

```
public frmBotonesRadio(String title) {
    super(title);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setContentPane(pnlImagenes);
    this.setMinimumSize(new Dimension(400, 400));
    this.setLocationRelativeTo(getParent());
}
```

67. Retorne a la clase **Principal** y dentro de su método **main**, convierta en comentario a las líneas que crean, instancian y hacen visible al objeto **formpersona**.

A cambio, inserte las siguientes líneas que crean una instancia de la clase **frmBotonesRadio** a probar ahora:

```
frmBotonesRadio frame = new frmBotonesRadio(
    "Selección de imágenes con radiobutton");
frame.setVisible(true);
```

68. Ejecute su aplicación y verifique que se abre la instancia del nuevo formulario para probar botones de radio.

Cierre la ventana para retornar a la vista de diseño.

69. Retorne a la clase del form **frmBotonesRadio**. Luego, por cada **JRadioButton** crearle a un Listener (*ActionListener*) de tipo *ActionPerformed*.

70. Ahora, agregue el siguiente código en los métodos *ActionPerformed* correspondientes creados en el paso anterior:

```
rbtOpcion1.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        lblImagen.setIcon(new
            javax.swing.ImageIcon(
                getClass().getResource("/interfaceswing/recursos/img1.jpeg")));
    }
});
rbtOpcion2.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        lblImagen.setIcon(new
            javax.swing.ImageIcon(
                getClass().getResource("/interfaceswing/recursos/img2.jpeg")));
    }
});
rbtOpcion3.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        ImageIcon imagen = new ImageIcon(
            getClass().getResource("/interfaceswing/recursos/img3.jpeg"));
        lblImagen.setIcon(imagen);
    }
});
```

71. Vuelva a ejecutar su aplicación y haga clic en los botones de radio. Verifique que se presenta una imagen distinta al presionar cada **JRadioButton**.

72. Guarde los cambios de su proyecto y llame a su instructor para mostrarle su funcionamiento.

IV. EJERCICIOS COMPLEMENTARIOS

Investigue cual es la utilidad de los archivos archivo CSV.

Como resultado de su investigación, en la carpeta **src** de su proyecto del procedimiento de esta practica, cree a un paquete llamado **datos** y dentro de este almacene a un archivo CSV que contenga a la información de al menos 3 personas, cuyas secuencias de datos sean compatibles con el modelo de tabla definido para el objeto **tblDatos**.

Luego, modifique la funcionalidad de la clase **frmPersona** de la siguiente forma:

- Crear un método privado que ubique y acceda al contenido del archivo CSV creado en el paquete **datos**. Solamente cuando exista este archivo, se procederá a cargar los registros almacenados ahí hacia el objeto **tblDatos** existente en el formulario. Este nuevo método deberá ser llamado solamente desde el método constructor.
- Crear un método privado, el cual recorra a todas las filas almacenadas en el objeto **tblDatos** y escriba sus datos hacia un archivo CSV, creado en el paquete **datos** de su proyecto.

Nota: Este archivo CVS a crear con este método es también el archivo utilizado por el método solicitado en el literal anterior de estas instrucciones.

- Crear un método privado que reciba como único parámetro a un vector de tipo Object con los datos de una Persona. Este método busca en las filas existentes en el objeto **tblDatos**. Si el registro de la persona recibida aun no existe en la tabla, inserta los datos del parámetro recibido como una nueva fila en el objeto **tblDatos**. De lo contrario, actualiza la fila coincidente con los valores recibidos en el parámetro.

Cada método solicitado en este análisis, sera evaluado solamente si es invocado en el procedimiento.

V. BIBLIOGRAFIA

- Hommel, S.(2007). Early Access Lesson: Learning Swing with the NetBeans IDE. Recuperado de <https://blogs.oracle.com/java/post/early-access-lesson-learning-swing-with-the-netbeans-ide>
- Ruiz, I. (2023). JavaHispano. JTable con Ejemplos-Parte I. Documento web: http://www.javahispano.org/contenidos/archivo/63/jtable_1.pdf
- ORACLE Java Documentation (2025). The Java TM Tutorials. Using Swing Components. Recuperado de <https://docs.oracle.com/javase/tutorial/uiswing/components/table.html>

VI. ANEXOS

Anexo 1: Falta diseñador de GUI con Frame

En caso de que el IDE IntelliJ-IDEA no cuente con el diseñador de interfaces graficas con JFrame, se deben ejecutar los siguientes pasos para instalar este complemento:

1. Cargar el IDE IntelliJ-IDEA
2. Hacer clic en el menu principal *File -> Setting*
3. Del listado de opciones a la izquierda, seleccionar a opcion *Plugins*.
4. En el contenido de opciones (a la derecha), ubicar en la parte superior a la ficha *Marketplace*.
5. En el cuadro de búsqueda, digite: `swing ui designer`
Al encontrar al plugins, instálelo.
6. Cuando el IDE complete la instalacion del complemento, se le solicitara reiniciar el entorno (Restart IDE) para que se apliquen los cambios en la configuracion.
7. Haga clic en esta opcion para reiniciar IDE.
Espere un momento hasta que se abra nuevamente el IDE.

Al retornar al proyecto, ya tendrá disponible el asistente para crear los GUI/formularios basados en JFrame.